

Hamiltonian Path in the Square of a Tree - CS215 Discrete Math (H) Project

Abstract

This report studies a traversal problem on a tree T with vertices $1, 2, \dots, N$: starting at vertex 1 and ending at vertex N , we must visit all vertices **exactly once**, where each step may move along a tree distance of 1 or 2. This is equivalent to finding a Hamiltonian path from 1 to N in the **square** of the tree T^2 .

A key point is that Hamiltonian paths in T^2 are determined by precise structural properties of T . We use two classical results: (i) Harary–Schwenk’s characterization of when T^2 has a Hamiltonian **cycle** (exactly when T is a **caterpillar**), and (ii) Radoszewski–Rytter’s characterization of when T^2 has a Hamiltonian **path** with prescribed endpoints, via **horsetail** trees. Based on these insights, we present an $O(N)$ algorithm: it extracts the unique tree path from 1 to N as a backbone, checks that each off-backbone component is a caterpillar and that the backbone satisfies the required “free-buffer” constraints, and then constructs the Hamiltonian path in T^2 by concatenating carefully chosen Hamiltonian subpaths inside each backbone layer. We also include a complete C++ implementation that outputs the traversal order or reports impossibility.

1. Problem Statement

Given a tree T with N vertices ($1 \leq N \leq 5 \times 10^5$), output an ordering of all vertices

$$v_1, v_2, \dots, v_N$$

such that:

1. $v_1 = 1$ (start at vertex 1),
2. $v_N = N$ (end at vertex N),
3. all vertices are distinct (each vertex visited exactly once),
4. for every i , the distance in the tree satisfies $\text{dist}_T(v_i, v_{i+1}) \in \{1, 2\}$.

If impossible, output **BRAK**.

2. Graph-Theoretic Formulation

2.1 Graph powers and the square of a graph

For a graph $G = (V, E)$, its **k -th power** G^k is the graph on the same vertex set V where distinct vertices u, v are adjacent in G^k iff their distance in G is at most k :

$$(u, v) \in E(G^k) \iff 1 \leq \text{dist}_G(u, v) \leq k.$$

In particular:

- G^2 is called the **square** of G ,
- G^3 is called the **cube** of G .

2.2 Equivalence to Hamiltonian path in T^2

In our problem, we may move from a vertex to another vertex at tree-distance 1 or 2. This is exactly the adjacency rule in T^2 . Therefore, the task is equivalent to:

Find a Hamiltonian path in T^2 from 1 to N .

Terminology used in this report:

- A **k -path** in G means a path in G^k .
- A **kH -path** in G means a Hamiltonian path in G^k .
- Similarly, **kH -cycle** means a Hamiltonian cycle in G^k .

Thus we seek a **2H-path** in the tree T , with fixed endpoints 1 and N .

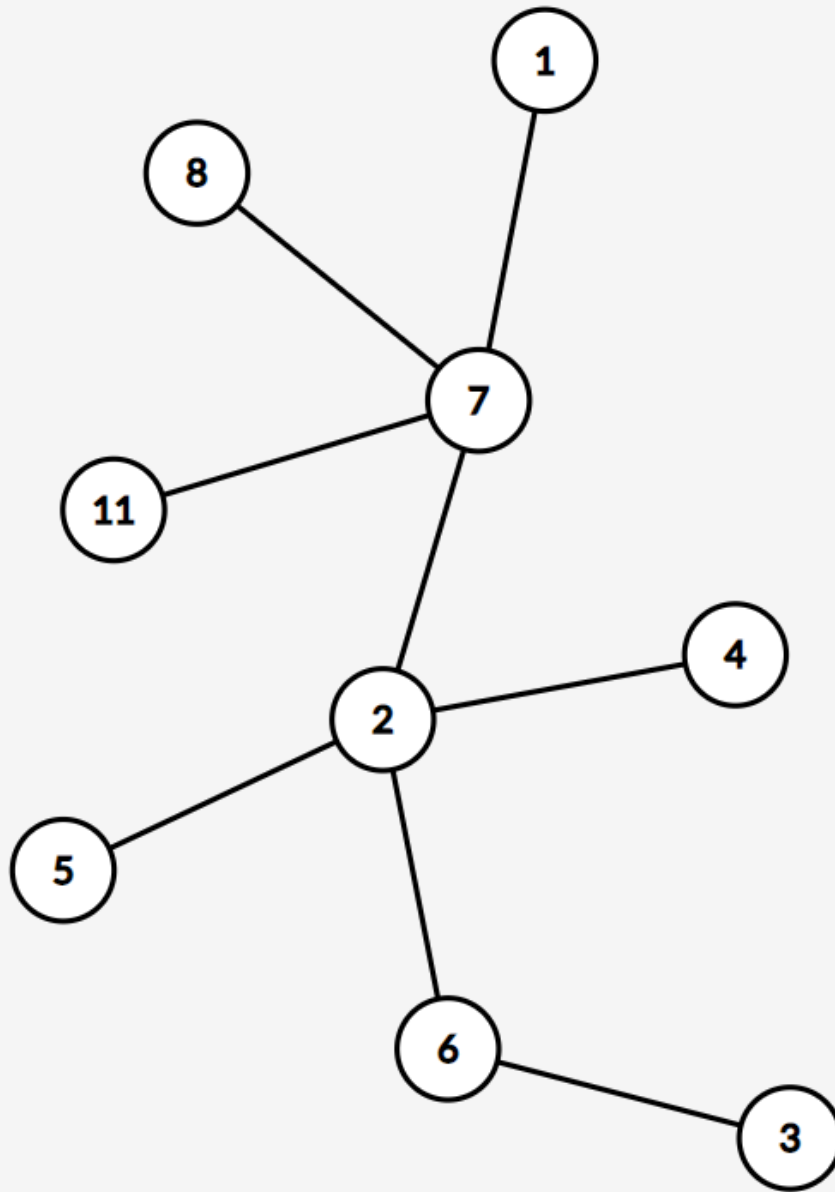
3. Structural Definitions: Caterpillars and Horsetails

3.1 Caterpillar

A **caterpillar** is a tree such that removing all leaves leaves a single simple path. That remaining path is called the **spine**.

Intuitively, a caterpillar is “a path with some leaves attached to spine vertices”.

Figure (example caterpillar):



We call a caterpillar **non-trivial** iff it has at least one edge (equivalently, at least two vertices).

3.2 Backbone path $P_{1 \rightarrow N}$ and layers

Because T is a tree, there is a unique simple path from 1 to N . Let this path be

$$u_1 = 1, u_2, \dots, u_L = N,$$

and call it the **backbone** (or main path).

For each backbone vertex u_i , consider all vertices in subtrees attached to u_i **excluding** the two backbone neighbors (if they exist). We treat the set consisting of u_i plus all its non-backbone attached vertices as the “layer” of u_i , denoted $\text{layer}(u_i)$.

We also classify vertices:

- **main**: backbone vertices u_i ,
- **secondary**: non-backbone vertices directly adjacent to some backbone vertex.

A fundamental “interface” limitation between neighboring layers will be used repeatedly:

Lemma 3.1 (Interface rule across adjacent layers).

Let u_i and u_{i+1} be adjacent on the backbone. If s is a secondary neighbor of u_i , and t is a secondary neighbor of u_{i+1} , then the unique tree path is

$$s - u_i - u_{i+1} - t,$$

so $\text{dist}_T(s, t) = 3$. Hence $(s, t) \notin E(T^2)$.

Consequence.

A valid step in T^2 that **crosses** from layer i to layer $i+1$ must involve at least one main vertex (either u_i or u_{i+1}). In particular, a Hamiltonian path in T^2 can never jump “secondary-to-secondary” across two adjacent backbone layers.

3.3 Horsetail

The paper gives an exact characterization of when T^2 has a Hamiltonian path with given endpoints $(1, N)$. The construction and checker in this report use the following operational definition along the backbone $P_{1 \rightarrow N}$.

Fix a backbone vertex u_i . Look at each connected component attached to u_i after removing the backbone edges (equivalently: each off-backbone subtree that meets the rest of T only at u_i).

1. Local caterpillar condition.

Every off-backbone component attached to u_i must be a caterpillar. Otherwise the instance is impossible.

2. At most two non-trivial attachments.

Let $\text{non_trivial}(u_i)$ be the number of attached caterpillars that are **non-trivial** (i.e., attached subtree is not just a single leaf).

If $\text{non_trivial}(u_i) > 2$, then impossible.

3. Type A / Type B.

- **Type A:** $\text{non_trivial}(u_i) \leq 1$,
- **Type B:** $\text{non_trivial}(u_i) = 2$.

4. Free vertex.

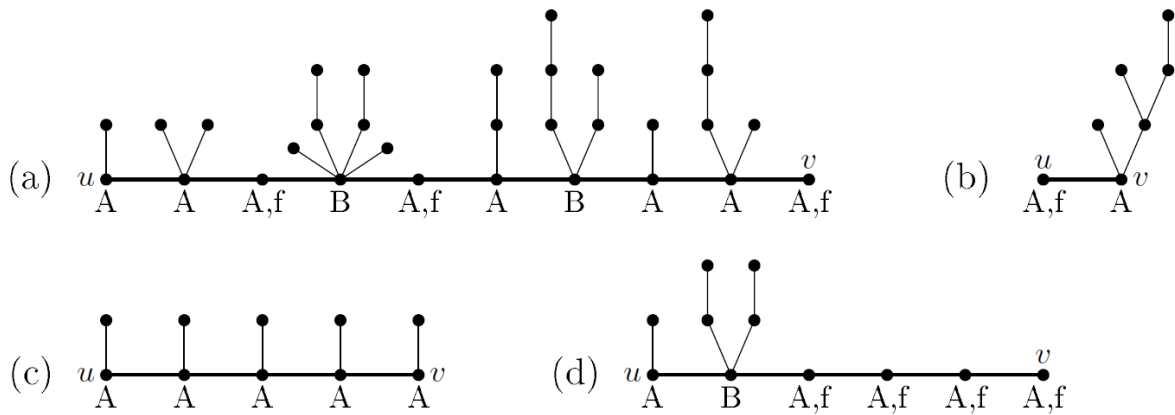
u_i is **free** iff it has no off-backbone vertices at all (i.e., $\text{layer}(u_i) = \{u_i\}$ only).

The backbone must satisfy these constraints (exactly what the implementation checks):

- (C2) Between any two Type B vertices on the backbone, there is at least one free vertex.
- (C3) Immediately before each Type B vertex, there is at least one free vertex.
- (C4) Immediately after each Type B vertex, there is at least one free vertex.
- (C5) There exists at least one free vertex on the backbone.

If all conditions hold, we call the tree a **(1, N)-horsetail** (in this fixed-endpoint setting).

Figure (illustrating free / Type B constraints):



4. Key Theoretical Results

4.1 Hamiltonian cycles in the square of a tree

Theorem 4.1 (Harary–Schwenk).

Let T be a tree with $|V(T)| \geq 3$. Then T^2 contains a Hamiltonian cycle **if and only if** T is a caterpillar.

Proof (sufficient direction)

Assume T is a caterpillar. Let the spine be a path $v_1, v_2, \dots, v_\ell$. For each spine vertex v_i , let P_i be an arbitrary ordering of its attached leaves (possibly empty).

A concrete Hamiltonian **cycle** in T^2 can be obtained by “interleaving” the spine vertices with leaf-blocks in a parity-dependent pattern (see §5). The key local check is always the same:

- two consecutive spine vertices are tree-distance 1;
- a leaf of v_i to v_i is distance 1;
- a leaf of v_i to a neighbor spine vertex $v_{i\pm 1}$ is distance 2;
- a leaf of v_i to a leaf of the same spine node is distance 2 via v_i .

Thus every consecutive pair in that cyclic ordering has tree-distance ≤ 2 , i.e., is an edge in T^2 . This gives a Hamiltonian cycle in T^2 .

Proof (necessary direction)

Assume T^2 has a Hamiltonian cycle C . Let T' be the tree obtained from T by deleting all leaves. A tree is a caterpillar iff T' is a (possibly trivial) path, i.e., T' has maximum degree ≤ 2 .

So it suffices to show that T' cannot contain a vertex r of degree at least 3. Suppose for contradiction that $r \in T'$ has three distinct neighbors $a, b, c \in T'$. Consider the three branches B_a, B_b, B_c in $T - r$ that contain a, b, c , respectively.

Key geometric fact: if $x \in B_a$ with $\text{dist}_T(r, x) \geq 2$, and $y \in B_b \cup B_c$, then the unique path from x to y must pass through r , so

$$\text{dist}_T(x, y) \geq \text{dist}_T(x, r) + \text{dist}_T(r, y) \geq 2 + 1 = 3,$$

meaning $(x, y) \notin E(T^2)$. Therefore, all vertices “deep” in each branch can connect to the outside world in T^2 only through the small interface

$$I := \{r\} \cup N_T(r),$$

and, even more tightly, only through vertices within tree-distance ≤ 2 from r .

Now look at the Hamiltonian **cycle** C in T^2 . For each deep branch B_a , the vertices deep in that branch form a nonempty set whose neighbors outside the branch all lie in I . Because C is a single cycle that visits each vertex exactly once, the cycle must “enter and leave” the deep region of each branch using interface vertices—creating at least one “attachment pair” (two cycle edges crossing between branch-region and outside). With three deep branches, we need (at least) three such attachment pairs.

But every interface vertex in a Hamiltonian cycle has degree exactly 2 in the cycle. When you try to realize three deep attachments through the **same small interface** I (consisting of r and its neighbors), pigeonhole-type constraints force either:

- some interface vertex to serve in more than two cross-region connections (impossible in a Hamiltonian cycle), or
- an attempted cross-connection that requires a distance- ≥ 3 step in T (not an edge of T^2).

Harary–Schwenk formalize this bottleneck argument and conclude such a branching vertex in T' is impossible. Hence $\Delta(T') \leq 2$, so T' is a path, so T is a caterpillar. $\square$

Note: The above is the standard proof **idea**; the original paper provides a concise formal argument.

4.2 Hamiltonian paths in the square of a tree (fixed endpoints)

We now focus on Hamiltonian **paths** in T^2 from 1 to N .

Theorem 4.2 (Radoszewski–Rytter, endpoint version).

Let T be a tree and let endpoints be 1 and N . Then T^2 contains a Hamiltonian path from 1 to N **if and only if** T is a $(1, N)$ -horsetail (as defined in §3.3).

This report's algorithm is essentially a direct implementation of the “if” direction (constructive) and a verification of the operational constraints that are forced by any solution.

We include the key necessity reasoning in §8.3 (completeness), so the correctness proof is self-contained with respect to the constraints actually checked by the program.

5. Constructing a Hamiltonian Cycle in a Caterpillar Square

Let the caterpillar spine be

$$v_1, v_2, \dots, v_\ell,$$

and let P_i be an ordering of the leaves attached to v_i .

One canonical cycle pattern (used by the blog and adapted in code) is:

- If ℓ is even:

$$v_1, P_2, v_3, \dots, v_{\ell-1}, P_\ell, v_\ell, P_{\ell-1}, \dots, P_3, v_2, P_1.$$

- If ℓ is odd:

$$v_1, P_2, v_3, \dots, P_{\ell-1}, v_\ell, P_\ell, v_{\ell-1}, \dots, P_3, v_2, P_1.$$

Why consecutive pairs are valid edges of T^2 .

In each transition, you are either:

- moving along the spine (distance 1), or
- moving between a leaf and its spine node / adjacent spine node (distance 1 or 2), or
- moving between two leaves of the same spine node (distance 2).

So every consecutive pair has tree-distance ≤ 2 .

Figure (cycle pattern illustration):



Fig. 5. To the left: a 2H-cycle $1, 2, \dots, 8$ in a caterpillar with the spine $1-3-7-5$ of even length. To the right: a 2H-cycle $1, 2, \dots, 10$ in a caterpillar with an odd spine $1-3-9-5-7$.

From cycle to path

To obtain a Hamiltonian **path**, we can “break” the Hamiltonian cycle by removing one edge, producing a Hamiltonian path whose endpoints are the two vertices that were adjacent on the removed edge. This is the main trick used inside each caterpillar-like layer in the final construction.

6. High-Level Algorithm for Horsetail Trees

6.1 Overview

- 1. Extract the backbone:** compute parent pointers by DFS from 1, then read off the unique path from N back to 1.
- 2. Validate horsetail structure:**
 - For each backbone node u_i , check each attached off-backbone subtree is a caterpillar.
 - Count non-trivial attached caterpillars; reject if > 2 .
 - Mark free nodes; enforce constraints (C2)–(C5).
- 3. Layer-by-layer construction:**
 - For each backbone node u_i , build a Hamiltonian path inside $\text{layer}(u_i)$ in T^2 , with carefully selected endpoints so consecutive layers can be concatenated with step distance ≤ 2 .
- 4. Output concatenation** of all layer paths to obtain a global Hamiltonian path in T^2 from 1 to N .

6.2 Why “free vertices” are needed

If the current layer ends at a **secondary** vertex adjacent to u_i , then the next layer cannot start at a secondary adjacent to u_{i+1} (distance would be 3); it must start at the main vertex u_{i+1} (or something within distance ≤ 2 that involves the main vertex). Conversely, if a layer must start at a secondary (which happens in Type B situations when the traversal needs to enter and leave through two distinct secondary “ports”), then the previous layer must end at a main vertex.

A **free** backbone vertex u_j (layer size 1) can both start and end at the main vertex, and therefore acts as a “buffer” to switch between “main endpoint” and “secondary endpoint” requirements while maintaining the distance- ≤ 2 boundary condition. This is exactly why constraints (C2)–(C4) exist.

7. Detailed Construction Components

7.1 Checking whether an attached subtree is a caterpillar

Consider an off-backbone subtree rooted at a child of u_i . The implementation checks caterpillar-ness using the following equivalent property:

In a caterpillar, after removing leaf children locally, each internal node has **at most one** non-leaf child along the remaining “core chain”.

Implementation view (`isCaterpillar(fa, p)`):

- From node p , ignore the parent and ignore degree-1 children (leaves).
- Recurse into the remaining children; if there are **two or more** such non-leaf children, the leaf-stripped core would branch, so it is not a caterpillar.

This is a linear-time check over the subtree.

7.2 Building a layer path: `BuildCaterpillar(mp, S, T)`

For a backbone node mp , the layer contains:

- the node mp ,
- its directly attached leaves,
- and up to 2 non-trivial caterpillars.

The routine:

1. Builds a “spine list” by traversing the non-trivial caterpillar cores; if there are two non-trivial caterpillars, it reverses one to glue both into one effective spine chain through mp .
2. Attaches leaves to their spine nodes in a separate adjacency structure (so leaves can be output in blocks).
3. Generates a Hamiltonian cycle ordering in the square of this caterpillar-like structure using the parity interleaving idea in §5.
4. Chooses a direction around the cycle and “breaks” at the prescribed endpoints S, T , carefully handling the case where S or T are leaves (by mapping leaf endpoints to their spine parent and emitting leaf-blocks before/after the spine traversal).

The output is a sequence that visits every vertex in the layer exactly once, starts at S , ends at T , and keeps each consecutive pair at tree-distance ≤ 2 .

7.3 Concatenating layers: `Get2HPathHorsetail()`

The global constructor processes backbone nodes in order:

$$u_1 = 1, u_2, \dots, u_L = N.$$

At each layer u_i , it chooses:

- a start endpoint fst ,
- and an end endpoint lst ,
where each is either the main vertex u_i or a secondary neighbor, so that:
- boundary hop from previous layer end to current layer start is within distance ≤ 2 ,
- and the selected end type (main vs secondary) makes the next step feasible.

Pseudocode snapshot reference:

Algorithm 2H-path-horsetail(T, u, v)

```
1: Compute the main nodes  $u_1, \dots, u_k$  and the layers  $Layer(u_1), \dots, Layer(u_k)$ ;
2:  $first := u_1$ ;
3: if  $free(u_1)$  then  $last := u_1$ 
4: else  $last := any-secondary-node(Layer(u_1))$ ;
5:  $S := 2H-path-caterpillar(Layer(u_1), first, last)$ ;
6: for  $i := 2$  to  $k$  do
7:    $w := u_i$ ;
8:   if  $w$  is a free node then
9:      $first := last := w$ ;
10:  else if  $last$  is a secondary node then
11:     $first := w$ ;
12:     $last := a-secondary-node-preferably-leaf(Layer(w))$ ;
13:  else
14:     $first := a-secondary-node-preferably-leaf(Layer(w))$ ;
15:     $last := w$ ;
16:  if  $2H-path-caterpillar(Layer(w), first, last) = \text{NONE}$  then
17:     $last := another-secondary-node(Layer(w))$ ;
18:   $S := S + 2H-path-caterpillar(Layer(w), first, last)$ ;
19: return  $S$ ;
```

8. Correctness Argument

We prove that the algorithm prints a valid traversal **if and only if** the input tree satisfies the $(1, N)$ -horsetail constraints in §3.3 (equivalently, if and only if a valid traversal exists).

Let the backbone be

$$u_1 = 1, u_2, \dots, u_L = N,$$

and the output be a concatenation of layer paths:

$$S = S_1 \circ S_2 \circ \dots \circ S_L,$$

where S_i is returned by `BuildCaterpillar(u_i, fst_i, lst_i)`.

8.1 Soundness

(i) Coverage and no repetition

Claim 8.1 (Layer partition).

The sets $\text{layer}(u_1), \dots, \text{layer}(u_L)$ form a partition of $V(T)$.

Reason.

Every vertex is either on the backbone or belongs to exactly one off-backbone component attached to a unique backbone vertex (the first backbone vertex encountered on the path to the backbone). Hence it belongs to exactly one layer.

Claim 8.2 (Each layer path is Hamiltonian on its layer).

For each i , `BuildCaterpillar(u_i, fst_i, lst_i)` outputs a sequence containing every vertex in $\text{layer}(u_i)$ exactly once and no vertex outside it.

Reason.

`BuildCaterpillar` explicitly constructs a Hamiltonian path inside the layer square by generating a caterpillar-cycle order and breaking it, while enumerating all leaves/cores attached to u_i . It never references vertices from other layers.

Combining Claims 8.1–8.2: S visits every vertex exactly once.

(ii) Adjacency constraint $\text{dist}_T(v_j, v_{j+1}) \in \{1, 2\}$

- Inside each S_i : consecutive vertices are adjacent in the square of the layer, so their tree-distance is ≤ 2 .
- Between layers: the last vertex of S_i is lst_i and the first vertex of S_{i+1} is fst_{i+1} . The endpoint selection logic in `Get2HPathHorsetail()` ensures $\text{dist}_T(\text{lst}_i, \text{fst}_{i+1}) \leq 2$, using exactly the allowed interface patterns implied by Lemma 3.1.

Therefore every step in S satisfies the distance condition.

(iii) Endpoints

By construction:

- $fst_1 = 1$,
- $lst_L = N$.

So the output starts at 1 and ends at N .

Thus the printed sequence is a valid solution. $\square$

8.2 Completeness

We show that each rejection condition corresponds to a necessary constraint for any Hamiltonian path in T^2 from 1 to N .

(a) Non-caterpillar off-backbone component implies impossibility

Fix a backbone vertex u_i and an attached component C hanging off u_i . Every path from any $x \in C$ to any vertex outside $C \cup \{u_i\}$ must pass through u_i .

If $x \in C$ has $\text{dist}_T(x, u_i) \geq 3$, then for any vertex $y \notin C$,

$$\text{dist}_T(x, y) \geq \text{dist}_T(x, u_i) + 1 \geq 4,$$

so x has **no** neighbors outside $C \cup \{u_i\}$ in T^2 . Hence all “deep” vertices of C must appear as a contiguous block in any Hamiltonian path of T^2 , and the block can connect to the rest only through a small neighborhood around u_i (within distance 2 from u_i).

If C is **not** a caterpillar, then its leaf-stripped core contains a branching vertex of degree ≥ 3 . That branching creates three deep directions inside C that must each be entered/exited through the same limited attachment neighborhood near u_i . A Hamiltonian **path** can only provide two “global ends” and cannot support three independent deep traversals without either:

- reusing some attachment-neighborhood vertex (violating Hamiltonian property), or
- requiring a forbidden distance- ≥ 3 jump.

Therefore any valid solution forces every attached component to be a caterpillar, so rejecting on this condition is correct.

(b) More than two non-trivial attached components at one backbone vertex implies impossibility

Suppose some backbone vertex u_i has three non-trivial attached caterpillars C_1, C_2, C_3 . In each C_j , pick a vertex x_j at distance at least 2 from u_i . As above, each x_j can only connect to the outside world through a narrow neighborhood near u_i . Thus in any Hamiltonian path, vertices of each C_j must be traversed in a contiguous block, and each block must attach to the rest of the path through the same small interface near u_i .

But three disjoint deep blocks require three “attachments” to be spliced into one global path. A Hamiltonian path can only have two ends overall and cannot splice three deep blocks through the same interface without repeating interface vertices. Hence $\text{non_trivial}(u_i) \leq 2$ is necessary.

So rejecting when $\text{non_trivial}(u_i) > 2$ is correct.

(c) Missing free buffers around Type B implies impossibility

Now assume u_i is Type B: it has exactly two non-trivial attached caterpillars. Covering both non-trivial attachments without repeating vertices forces the traversal within layer i to “use” two distinct secondary-side access points (informally: one to enter one non-trivial part and one to exit after finishing the other), so the layer path typically must start/end at **secondary** vertices.

Combine this with Lemma 3.1:

- If layer i ends at a secondary of u_i , then layer $i+1$ cannot start at a secondary of u_{i+1} ; it must start at the main u_{i+1} .
- If layer i must start at a secondary, then layer $i-1$ must end at a main vertex.

A **free** vertex layer $\{u_j\}$ can start and end at the main vertex, which is exactly what allows these endpoint-type switches without creating a forbidden secondary-to-secondary boundary.

Therefore:

- there must be at least one free vertex immediately before each Type B,

- at least one free vertex immediately after each Type B,
- and at least one free vertex between any two Type B vertices,
- plus at least one free vertex overall to make the endpoint-type parity feasible from 1 to N .

These are exactly (C2)–(C5). Hence rejecting when they fail is correct.

Since every possible rejection condition corresponds to a necessary constraint, if `Check()` rejects, no valid traversal exists. $\square$

9. Complexity

Each vertex and edge is processed $O(1)$ times across:

- parent/backbone extraction,
- caterpillar checks on disjoint off-backbone subtrees,
- layer construction and concatenation.

Thus the total time complexity is $O(N)$, and memory usage is $O(N)$.

10. Implementation Notes

- The implementation uses adjacency lists and parent pointers to extract the backbone.
 - Caterpillar checks and layer traversals are done in linear total work over the entire tree.
 - Warning: recursive DFS on a path-like tree of length 5×10^5 may overflow stack depending on environment. (The provided code uses recursion as commonly accepted on some judges; in strict environments an iterative DFS is safer.)
-

11. Conclusion

This project shows how a seemingly simple constraint on a tree walk becomes substantially more structured once we allow steps of tree-distance 2, because the task is exactly a Hamiltonian path problem in the square graph T^2 . The existence of such a path is not arbitrary: it is governed by a precise tree shape (the $(1, N)$ -horsetail). This structural view turns the problem from “searching for a Hamiltonian path” into a deterministic linear-time procedure.

On the theoretical side, the key insight is that local branching cannot be too complex: every off-backbone component must be a caterpillar and each backbone vertex can have at most two non-trivial attached components. On the global side, feasibility is controlled by a concrete distance barrier: by Lemma 3.1, a secondary vertex of u_i cannot jump directly to a secondary vertex of u_{i+1} because the tree-distance is 3. Therefore, when a Type B layer forces the traversal to use secondary endpoints, the backbone must contain enough **free** vertices to buffer endpoint-type transitions (main vs secondary) without violating the distance- ≤ 2 rule. These constraints are exactly what `Check()` verifies.

Algorithmically, once the horsetail structure is confirmed, the construction becomes clean: we traverse the backbone from 1 to N , build a Hamiltonian path inside each layer using the caterpillar “cycle-then-break” technique, and concatenate layer paths while respecting the interface rule. Because each vertex belongs to exactly one layer and each layer is processed in linear time in its size, the overall time and memory complexities are $O(N)$, meeting the problem’s large-input requirements.

References

1. Frank Harary, Allen Schwenk, “**Trees with Hamiltonian square.**” *Mathematika* 18(1): 138–140, 1971. DOI: <https://doi.org/10.1112/S0025579300008494>
Cambridge page: <https://www.cambridge.org/core/journals/mathematika/article/abs/trees-with-hamiltonian-square/D9E209DAD13C245ED72E73E57BEA53DF>
<https://www.semanticscholar.org/paper/Trees-with-Hamiltonian-square-Harary-Schwenk/7afbc37ac4c12626de755865530b46301a86b39e>
2. Jakub Radoszewski, Wojciech Rytter, “**Hamiltonian Paths in the Square of a Tree.**” In:

Algorithms and Computation (ISAAC 2011), LNCS 7074, pp. 90–99, Springer, 2011.

Springer chapter: https://link.springer.com/chapter/10.1007/978-3-642-25591-5_11

PDF mirror: <http://tsawke.com/Data/File/content/Hamiltonian%20paths%20in%20the%20square%20of%20a%20tree.pdf>

3. Problem statement: Luogu P3549 — **[POI2013] MUL – Multidrink**

<https://www.luogu.com.cn/problem/P3549>

4. Original blog (my own) — **LG-P3549 [POI2013]MUL-Multidrink Solution**

<https://www.luogu.com.cn/article/pzvwkubh>

<http://tsawke.com/Data/Blog/content/LG-P3549-Solution.html>

Appendix. C++ Implementation

```
#define _USE_MATH_DEFINES
#include <bits/extc++.h>

#define PI M_PI
#define E M_E
#define npt nullptr
#define SON i->to
#define OPNEW void* operator new(size_t)
#define ROPNEW(arr) void* Edge::operator new(size_t){static Edge* P = arr;
return P++;}

/*****
abbr
mp => mainp
subt => subtree
fa => father
fst => first
lst => last
*****/

using namespace std;
```

```

using namespace __gnu_pbds;

mt19937 rnd(random_device{}());
int rndd(int l, int r){return rnd() % (r - l + 1) + l;}
bool rnddd(int x){return rndd(1, 100) <= x;}

typedef unsigned int uint;
typedef unsigned long long unll;
typedef long long ll;
typedef long double ld;

#define EXIT puts("BRAK"), exit(0)
#define MAXN 510000

template<typename T = int>
inline T read(void);

struct Edge{
    Edge* nxt;
    int to;
    OPNEW;
}ed[(MAXN << 1) + MAXN];
ROPNEW(ed);
Edge* head[MAXN];

int N;
int mainLen(0);
int fa[MAXN], deg[MAXN];
int mainp[MAXN];
int non_trivial[MAXN];
bool isMainp[MAXN], isFree[MAXN];

void dfs(int p = 1){
    for(auto i = head[p]; i; i = i->nxt)
        if(SON != fa[p])
            fa[SON] = p,
            dfs(SON);
}

void InitMainp(void){
    int cur = N;

```

```

do{
    mainp[++mainLen] = cur;
    isMainp[cur] = true;
    cur = fa[cur];
}while(cur != 1);
mainp[++mainLen] = 1;
isMainp[1] = true;
reverse(mainp + 1, mainp + mainLen + 1);
}

bool isCaterpillar(int fa, int p){
    int cnt(0);
    for(auto i = head[p]; i; i = i->nxt){
        if(SON == fa || deg[SON] == 1)continue;
        if(!isCaterpillar(p, SON))return false;
        if(cnt++)return false;
    }return true;
}

void Check(void){
    for(int p = 1; p <= mainLen; ++p){
        int mp = mainp[p];
        isFree[mp] = true;
        for(auto i = head[mp]; i; i = i->nxt){
            if(isMainp[SON])continue;
            isFree[mp] = false;
            if(deg[SON] == 1)continue;
            ++non_trivial[mp];
            if(non_trivial[mp] > 2)EXIT;
            if(!isCaterpillar(mp, SON))EXIT;
        }
    }
    int curFree(0);
    bool end_with_B(true);
    bool exist_free(false);
    for(int p = 1; p <= mainLen; ++p){
        int mp = mainp[p];
        if(isFree[mp]){++curFree; exist_free = true; end_with_B = false;
continue;}
        if(non_trivial[mp] == 2){
            if(!curFree)EXIT;
            curFree = 0;

```

```

        end_with_B = true;
    }
}
if(end_with_B || !exist_free)EXIT;
}
int FindAnySecondaryNode(int p){
    for(auto i = head[p]; i; i = i->nxt)
        if(!isMainp[SON])return SON;
    return -1;
}
int FindAnySecondaryNode_PreferablyLeaf(int p){
    for(auto i = head[p]; i; i = i->nxt)
        if(!isMainp[SON] && deg[SON] == 1)return SON;
    return FindAnySecondaryNode(p);
}
int FindAnotherSecondaryNode(int p, int lst){
    for(auto i = head[p]; i; i = i->nxt)
        if(!isMainp[SON] && SON != lst)return SON;
    return -1;
}
namespace Caterpillar{
    vector < int > route;
    Edge* head[MAXN];
    vector < int > spine;
    enum type{spineNode = 1, leafNode};
    int ffa[MAXN];
    void add(int s, int t){
        head[s] = new Edge{head[s], t};
        ffa[t] = s;
    }
    void BuildSpine(int fa, int p){
        spine.push_back(p);
        for(auto i = ::head[p]; i; i = i->nxt){
            if(SON == fa)continue;
            if(::deg[SON] == 1)add(p, SON);
            else BuildSpine(p, SON);
        }
    }
    void extend(int mp, int unreachable1 = -1, int unreachable2 = -1){
        for(auto i = head[mp]; i; i = i->nxt){

```

```

        if(SON == unreachable || SON == unreachable2)continue;
        route.push_back(SON);
    }
}

vector < int > BuildCaterpillar(int mp, int S, int T){
    route.clear();
    spine.clear();
    route.push_back(S);
    if(S == T)return route;
    spine.push_back(mp);
    bool exist_caterpillar(false);
    for(auto i = ::head[mp]; i; i = i->nxt){
        if(isMainp[SON])continue;
        if(deg[SON] == 1)add(mp, SON);
        else{
            if(!exist_caterpillar)exist_caterpillar = true;
            else reverse(spine.begin(), spine.end());
            BuildSpine(mp, SON);
        }
    }
    vector < pair < int, type > /*spine_node_pos, spine or leaf*/ >
temp;
    vector < pair < int, type > /*spine_node_pos, spine or leaf*/ >
unextended;
    for(int i = 0; i < (int)spine.size(); ++i)
        temp.push_back({spine.at(i), !(i & 1) ? spineNode :
leafNode});
    for(int i = (int)spine.size() - 1; i >= 0; --i)
        temp.push_back({spine.at(i), (i & 1) ? spineNode : leafNode});
    for(auto it = temp.begin(); it < temp.end(); ++it)
        if(it->second == spineNode || head[it-
>first])unextended.push_back(*it);
    #define LEFT(x) (x == 0 ? (int)unextended.size() - 1 : x - 1)
    #define RIGHT(x) (x == (int)unextended.size() - 1 ? 0 : x + 1)
    auto Beg = deg[S] == 1 ? make_pair(ffa[S], leafNode) :
make_pair(S, spineNode);
    auto End = deg[T] == 1 ? make_pair(ffa[T], leafNode) :
make_pair(T, spineNode);
    int begPos = -1; while(unextended.at(++begPos) != Beg);
    if(Beg.second == leafNode)extend(Beg.first, S, T);

```

```

        if(unextended.at(LEFT(begPos)) == End)
            for(int j = RIGHT(begPos); unextended.at(j) != End; j =
RIGHT(j))
                unextended.at(j).second == spineNode
                    ? route.push_back(unextended.at(j).first)
                    : extend(unextended.at(j).first);
            else
                for(int j = LEFT(begPos); unextended.at(j) != End; j =
LEFT(j))
                    unextended.at(j).second == spineNode
                        ? route.push_back(unextended.at(j).first)
                        : extend(unextended.at(j).first);
                if(End.second == leafNode && Beg != End)extend(End.first, S, T);
                route.push_back(T);
                return route;
    }
}

vector < int > Get2HPathHorsetail(void){
    vector < int > ret;
    int fst = mainp[1];
    int lst = isFree[mainp[1]]
        ? mainp[1]
        : FindAnySecondaryNode(mainp[1]);
    auto tmp = Caterpillar::BuildCaterpillar(mainp[1], fst, lst);
    ret.insert(ret.end(), tmp.begin(), tmp.end());
    for(int i = 2; i <= mainLen; ++i){
        int w = mainp[i];
        if(isFree[w])fst = lst = w;
        else if(!isMain[lst])
            fst = w,
            lst = FindAnySecondaryNode_PreferablyLeaf(w);
        else
            fst = FindAnySecondaryNode_PreferablyLeaf(w),
            lst = non_trivial[w] == 2
                ? FindAnotherSecondaryNode(w, fst)
                : w;
        auto cp = Caterpillar::BuildCaterpillar(w, fst, lst);
        ret.insert(ret.end(), cp.begin(), cp.end());
    }
    return ret;
}

```

```

}
int main(){
    N = read();
    for(int i = 1; i <= N - 1; ++i){
        int s = read(), t = read();
        head[s] = new Edge{head[s], t};
        head[t] = new Edge{head[t], s};
        ++deg[s], ++deg[t];
    }
    dfs();
    InitMainp();
    Check();
    auto ans = Get2HPathHorsetail();
    for(auto i : ans)printf("%d\n", i);
    return 0;
}

template<typename T>
inline T read(void){
    T ret(0);
    short flag(1);
    char c = getchar();
    while(c != '-' && !isdigit(c))c = getchar();
    if(c == '-')flag = -1, c = getchar();
    while(isdigit(c)){
        ret *= 10;
        ret += int(c - '0');
        c = getchar();
    }
    ret *= flag;
    return ret;
}

```